

Modül 17

Testing

Unit, Integration, E2E

Modül: 17 / 30

Ders Sayısı: 8 ders

Seviye: İleri

Versiyon: Angular v21

Hazırlayan: Claude • Türkçe Angular v21 Eğitimi

Modül 17'ye Hoş Geldin

Test yazmak — profesyonel yazılım geliştirmenin olmazsa olmazı. Bu modülde modern Angular test stack'i (Jest/Vitest), TestBed kullanımı, component/service test pattern'leri ve E2E testing'i (Playwright) öğreneceğiz.

Öğrenecekler:

- ✓ Modern Angular test setup (Karma yerine Jest/Vitest)
- ✓ TestBed ile component test
- ✓ Service test (HttpTestingController)
- ✓ Pipe test
- ✓ Directive test
- ✓ Mock & Spy
- ✓ Signal'lı component test
- ✓ E2E testing — Playwright

Modern Test Setup

Karma'dan Jest/Vitest'e geçiş.

⚙️ Jest Setup (Önerilen)

```
npm install -D @jest/globals jest @types/jest \
  jest-environment-jsdom jest-preset-angular

# jest.config.js
module.exports = {
  preset: 'jest-preset-angular',
  setupFilesAfterEach: ['<rootDir>/setup-jest.ts'],
  testEnvironment: 'jsdom',
  testMatch: ['**/+(*)+(spec).+(ts)'],
};

# setup-jest.ts
import 'jest-preset-angular/setup-jest';
```

⚡ Vitest Setup (Modern)

```
npm install -D vitest @vitest/ui jsdom \
  @analogjs/vitest-angular

# vitest.config.ts
import { defineConfig } from 'vitest/config';
import angular from '@analogjs/vite-plugin-angular';

export default defineConfig({
  plugins: [angular()],
  test: {
    environment: 'jsdom',
    setupFiles: ['./src/test-setup.ts'],
  },
});
```

📄 Test Çalıştırma

```
# Jest
npm test                # Watch mode
npm test -- --coverage  # Coverage
npm test -- --runInBand # Sequential

# Vitest
npx vitest              # Watch
npx vitest run          # Tek seferlik
npx vitest --ui         # UI mode (çok güzel)
```

Component Test — TestBed

Component'leri izole şekilde test etmek.

□ Basic Component Test

```
import { ComponentFixture, TestBed } from '@angular/core/testing';
import { CounterComponent } from './counter.component';

describe('CounterComponent', () => {
  let component: CounterComponent;
  let fixture: ComponentFixture<CounterComponent>;

  beforeEach(async () => {
    await TestBed.configureTestingModule({
      imports: [CounterComponent], // Standalone
    }).compileComponents();

    fixture = TestBed.createComponent(CounterComponent);
    component = fixture.componentInstance;
    fixture.detectChanges();
  });

  it('should create', () => {
    expect(component).toBeTruthy();
  });

  it('should increment count', () => {
    component.increment();
    expect(component.count()).toBe(1);
  });

  it('should render count in template', () => {
    component.count.set(5);
    fixture.detectChanges();

    const el = fixture.nativeElement as HTMLElement;
    expect(el.textContent).toContain('5');
  });
});
```

□ Event Test

```
it('should increment on click', () => {
  const button = fixture.nativeElement.querySelector('button.increment');
  button.click();
  fixture.detectChanges();

  expect(component.count()).toBe(1);
});

// Veya dispatchEvent
it('should handle input', () => {
  const input: HTMLInputElement = fixture.nativeElement.querySelector('input');
  input.value = 'hello';
  input.dispatchEvent(new Event('input'));
  fixture.detectChanges();

  expect(component.name()).toBe('hello');
});
```

Service Test — HttpTestingController

HTTP isteklerini mock'lamak.

□ HTTP Mock Setup

```
import { TestBed } from '@angular/core/testing';
import { provideHttpClient } from '@angular/common/http';
import { provideHttpClientTesting, HttpTestingController } from
 '@angular/common/http/testing';

describe('UserService', () => {
  let service: UserService;
  let httpMock: HttpTestingController;

  beforeEach(() => {
    TestBed.configureTestingModule({
      providers: [
        provideHttpClient(),
        provideHttpClientTesting(),
        UserService,
      ],
    });

    service = TestBed.inject(UserService);
    httpMock = TestBed.inject(HttpTestingController);
  });

  afterEach(() => {
    httpMock.verify(); // Hiç pending request kalmamış
  });

  it('should fetch users', () => {
    const mockUsers = [{ id: '1', name: 'John' }];

    service.getUsers().subscribe(users => {
      expect(users).toEqual(mockUsers);
    });

    const req = httpMock.expectOne('/api/users');
    expect(req.request.method).toBe('GET');
    req.flush(mockUsers);
  });

  it('should handle 404', () => {
    service.getUsers().subscribe({
      error: err => expect(err.status).toBe(404)
    });

    httpMock.expectOne('/api/users').flush('Not found', {
      status: 404,
      statusText: 'Not Found'
    });
  });
});
```


Pipe Test

Pipe'lar pure function, test'i kolay.

❏ Pipe Test

```
import { TruncatePipe } from './truncate.pipe';

describe('TruncatePipe', () => {
  let pipe: TruncatePipe;

  beforeEach(() => {
    pipe = new TruncatePipe();
  });

  it('should truncate long string', () => {
    const result = pipe.transform('Hello World', 5);
    expect(result).toBe('Hello...');
  });

  it('should not truncate short string', () => {
    const result = pipe.transform('Hi', 5);
    expect(result).toBe('Hi');
  });

  it('should handle empty', () => {
    expect(pipe.transform('', 5)).toBe('');
    expect(pipe.transform(null as any, 5)).toBe('');
  });

  it('should use custom ellipsis', () => {
    expect(pipe.transform('Hello World', 5, '...')).toBe('Hello...');
  });
});
```

Directive Test

Test host component yaratıp directive'i test etmek.

□ Test Host Pattern

```
import { Component } from '@angular/core';
import { HighlightDirective } from './highlight.directive';

@Component({
  template: `<div [appHighlight]="color">Test</div>`,
  imports: [HighlightDirective],
})
class TestHostComponent {
  color = 'yellow';
}

describe('HighlightDirective', () => {
  let fixture: ComponentFixture<TestHostComponent>;

  beforeEach(async () => {
    await TestBed.configureTestingModule({
      imports: [TestHostComponent],
    }).compileComponents();

    fixture = TestBed.createComponent(TestHostComponent);
    fixture.detectChanges();
  });

  it('should apply background color', () => {
    const div = fixture.nativeElement.querySelector('div');
    expect(div.style.backgroundColor).toBe('yellow');
  });

  it('should update on color change', () => {
    fixture.componentInstance.color = 'blue';
    fixture.detectChanges();

    const div = fixture.nativeElement.querySelector('div');
    expect(div.style.backgroundColor).toBe('blue');
  });
});
```

Mock & Spy

Dependency'leri mock'lamak.

□ Service Mock

```
// Mock class
class MockUserService {
  getUsers = jest.fn().mockReturnValue(of([
    { id: '1', name: 'Mock User' }
  ]));

  deleteUser = jest.fn().mockReturnValue(of(void 0));
}

describe('UserListComponent', () => {
  let component: UserListComponent;
  let userService: MockUserService;

  beforeEach(async () => {
    await TestBed.configureTestingModule({
      imports: [UserListComponent],
      providers: [
        { provide: UserService, useClass: MockUserService }
      ],
    }).compileComponents();

    component = TestBed.createComponent(UserListComponent).componentInstance;
    userService = TestBed.inject(UserService) as any;
  });

  it('should call deleteUser', () => {
    component.delete('1');

    expect(userService.deleteUser).toHaveBeenCalledWith('1');
    expect(userService.deleteUser).toHaveBeenCalledTimes(1);
  });
});
```

□ Spy on Method

```
it('should spy on method', () => {  
  const spy = jest.spyOn(component, 'reloadData');  
  
  component.refresh();  
  
  expect(spy).toHaveBeenCalled();  
});  
  
// Mock return  
jest.spyOn(service, 'getData').mockReturnValue(of(['mocked']));
```

Signal Component Test

Modern signal-based component'leri test etmek.

✦ Signal Test Pattern

```

@Component({
  selector: 'app-counter',
  template: `
    <p>Count: {{ count() }}</p>
    <p>Doubled: {{ doubled() }}</p>
    <button (click)="increment()">+</button>
  `
})
class CounterComponent {
  count = signal(0);
  doubled = computed(() => this.count() * 2);

  increment() { this.count.update(c => c + 1); }
}

describe('CounterComponent', () => {
  let component: CounterComponent;
  let fixture: ComponentFixture<CounterComponent>;

  beforeEach(async () => {
    await TestBed.configureTestingModule({
      imports: [CounterComponent],
    }).compileComponents();

    fixture = TestBed.createComponent(CounterComponent);
    component = fixture.componentInstance;
    fixture.detectChanges();
  });

  it('should compute doubled', () => {
    component.count.set(5);
    fixture.detectChanges();

    expect(component.doubled()).toBe(10);
    expect(fixture.nativeElement.textContent).toContain('Doubled: 10');
  });

  it('should increment via button', () => {
    fixture.nativeElement.querySelector('button').click();
    fixture.detectChanges();

    expect(component.count()).toBe(1);
    expect(component.doubled()).toBe(2);
  });
});

```

E2E Testing — Playwright

Real browser testing.

⚙ Setup

```
npm init playwright@latest

# Test örneği
# tests/login.spec.ts
import { test, expect } from '@playwright/test';

test('login flow', async ({ page }) => {
  await page.goto('http://localhost:4200/login');

  await page.getByLabel('Email').fill('user@example.com');
  await page.getByLabel('Password').fill('password123');
  await page.getByRole('button', { name: 'Login' }).click();

  await expect(page).toHaveURL('/dashboard');
  await expect(page.getByText('Welcome')).toBeVisible();
});
```

□ Common Patterns

```
// API mock
test('mock API', async ({ page }) => {
  await page.route('/api/users', route => {
    route.fulfill({
      status: 200,
      body: JSON.stringify([ { id: '1', name: 'Mock' } ])
    });
  });

  await page.goto('/users');
  await expect(page.getByText('Mock')).toBeVisible();
});

// Wait for specific element
await page.waitForSelector('.spinner', { state: 'hidden' });

// Visual regression
await expect(page).toHaveScreenshot('homepage.png');

// Accessibility
import AxeBuilder from '@axe-core/playwright';

test('ally', async ({ page }) => {
  await page.goto('/');
  const results = await new AxeBuilder({ page }).analyze();
  expect(results.violations).toEqual([]);
});
```

□ Test Pyramid

	Sayı	Hız	Kapsam
	Çok	Hızlı (ms)	Function/method
Integration	Orta	Orta (saniye)	Component + service
	Az	Yavaş (dakika)	Tüm flow

Modül 17 Özeti

Testing artık alet kutunda. Unit'ten E2E'ye, mock'tan signal test'e — profesyonel test suite kurabilirsin.

Neler Öğrendin?

- ✓ Modern test setup (Jest/Vitest)
- ✓ Component test — TestBed
- ✓ Service test — HttpTestingController
- ✓ Pipe test
- ✓ Directive test (test host pattern)
- ✓ Mock & Spy
- ✓ Signal component test
- ✓ E2E — Playwright

Sıradaki Modül

Modül 18 — Component Communication Patterns. Parent-child, sibling, cross-tree communication patterns.